Programlama Dilleri

Dr. Öğr. Üyesi Hayri Volkan Agun

Bilgisayar Mühendisliği Bölümü

Bursa Teknik Üniversitesi

Halting Problem

- Halting problemi diyorki bir programın sonlanıp sonlanmayacağına karar verebilen başka bir program yazılamaz. Derleyiciler programın çalışıp sonlanacağına bu programı çalıştırmadan karar veremezler. Bu durumda program çalıştırılırsa ve sonlanmaz ise kendisi ile çelişen bir kararsızlık durumu oluşacaktır.

Halting Problem

- Belirli bir girdi verildiğinde bir programın döngüye girip girmeyeceğini keşfedebilen bir statik analizörün var olup olmadığını soracağız. Böyle bir denetimin ne kadar yararlı olduğunu vurgulamaya gerek yoktur. Eğer bir programın, belirli bir girdiyle çalıştırıldığında döngüye gireceğini çalıştırmadan önce biliyorsak, bu programın kesinlikle bir hata içerdiğini otomatik olarak gösterebiliriz.
- Statik analizör, derleyicinin bir alt programı olarak kullanılan bir programdan başka bir şey değildir. Bu nedenle, programları yazacağımız bir programlama dili L'yi sabitleyelim. Bir P programı ve bir x girdisi verildiğinde, P(x) ifadesi P'nin x üzerinde çalıştırılmasının sonucunu gösterir. Genel olarak P programının x üzerinde bir döngüye ya da sonsuz özyinelemeye (infinite recursion) girmesi nedeniyle sonlanmayabileceğini belirtmeliyiz. Dolayısıyla P(x) yazdığımızda, her zaman sonlanan bir hesaplamayı ifade etmiş olmayız. Genelliği kaybetmeden, soruyu şu şekilde yeniden formüle edebiliriz:
- L dilinde yazılmış bir P programı ve onun girdisi x verildiğinde, P(x) sonlanıyorsa sonlanıp "yes" yazdıran; P(x) döngüye giriyorsa yine sonlanıp "no" yazdıran bir H programı var mıdır?

Halting Problem

- Şimdi asıl amacımızı ortaya koyalım. Az önce tanımladığımız davranışa sahip bir H programının var olmadığını göstermek istiyoruz. Bunu çelişki yöntemiyle yapacağız: Böyle bir H'nin var olduğunu varsayacak ve bu varsayımdan bir çelişki türeteceğiz. İlk bakışta akıl yürütme biraz dolambaçlı görünebilir, ancak ileri düzey bilgi gerektirmez.
- Çelişki yoluyla, yukarıda belirtilen özelliklere sahip bir H programının var olduğunu varsayalım.
- H'yi kullanarak, tek girdisi başka bir program olacak şekilde K adlı bir program yazabiliriz. K'nın tanımı şu şekildedir:
- K programı, girdi olarak P verildiğinde, eğer H(P,P) "no" yazdırırsa sonlanıp "yes" yazdırır; eğer H(P,P) "yes" yazdırırsa sonsuz döngüye girer.
- H'nin elimizde olduğunu varsayarsak K'yı yazmak basittir. Girdi olarak P okunur, H bir alt program olarak çağrılır ve P hem birinci hem ikinci parametre olarak verilir. H'nin sonlanması beklenir (H'nin tanımına göre bu kesinlikle gerçekleşecektir). Eğer H(P,P) sonlanır ve "no" yazdırırsa (bu çıktıyı yakalayıp ekrana yansıtılmasını engelleyebileceğimizi varsayabiliriz), K "yes" yazdırır. Buna karşılık H(P,P) "yes" yazdırırsa, K bu amaçla programlanmış bir sonsuz döngüye girer.
- H'nin tanımını hatırlarsak, K'nın anlambilimini şu şekilde özetleyebiliriz:

  K(P) = "yes", eğer P(P) sonlanmazsa,

  sonlanmaz, eğer P(P) sonlanırsa.
- İlk bakışta bir programın kendisine uygulanması (P(P)) garip görünebilir. Ancak burada olağanüstü bir durum yoktur. P, kendi metnini temsil eden bir karakter dizisini girdi olarak alacaktır.

Halting Problemi

- Şimdi K'yı kendi metni üzerinde çalıştıralım; yani K(K) ifadesini inceleyelim. Davranışı nedir?
- P yerine K koyarsak şu sonucu elde ederiz:

  K(K) =“yes”, eğer K(K) sonlanmazsa,

  sonlanmaz, eğer K(K) sonlanırsa.
- Bunun saçma (absürt) olduğunu gözlemleyelim. Bu ifade, K(K)'nin sonlanmadığı durumda sonlanıp “yes” yazdırdığını; sonlandığı durumda ise sonlanmadığını söylemektedir. Bu açık bir çelişkidir.
- Bu çelişki nereden kaynaklanmaktadır? K'nın kendisinden değil; çünkü gördüğümüz gibi K, yalnızca H'yi kullanan basit bir programdır. Çelişki, H'nin belirtilen özelliklere sahip bir program olarak var olduğunu varsaymamızdan doğmaktadır. Dolayısıyla, böyle bir H programı var olamaz.

Kısıtlı Halting Problemi

- Bir programın sabit (constant) bir fonksiyon hesaplayıp hesaplamadığını doğrulama probleminin karar verilemez (undecidable) olduğunu gösteriniz.
- **İpucu:** Verilen herhangi bir P programı için, tek girdili yandaki Q_p programını tanımlayın:

$$Q_P(y) = \begin{cases} 1 & \text{eğer } P(P) \text{ sonlanırsa} \\ \text{sonlanmaz} & \text{aksi halde} \end{cases}$$

Kısıtlı Halting Problemi

- Bu ispat indirgeme (reduction) yöntemiyle yapılır; yani problemi durma problemine indirgeriz.
- Program

  QP(y): P'yi kendi girdisiyle çalıştır: P(P)

  Eğer P(P) sonlanırsa: 1 döndür

  Aksi halde: sonsuz döngüye gir
- Burada dikkat edilirse Q_p'nun çıktısı y'ye bağlı değildir; yalnızca P(P)'nin davranışına bağlıdır.

Kısıtlı Halting Problemi

- Eğer P(P) sonlanıyorsa: Q_p, hangi y girdisi verilirse verilsin P(P)'nin sonlandığını görür ve 1 çıktısını üretir. Dolayısıyla Q_p, tüm girdiler için 1 değerini üretir. Bu durumda Q_p **sabit 1 fonksiyonunu** hesaplar.
- Eğer P(P) sonlanmıyorsa: Q_p her y girdisinde P(P)'nin bitmesini bekler fakat bu asla gerçekleşmez. Dolayısıyla Q_p hiçbir girdi için sonlanmaz. Bu durumda Q_p hiçbir yerde tanımlı değildir (tamamen kısmi ve boş fonksiyon).
- Yani Q_p'nun sabit fonksiyon hesaplayıp hesaplamadığı, doğrudan P(P)'nin durma durumuna eşdeğerdir.

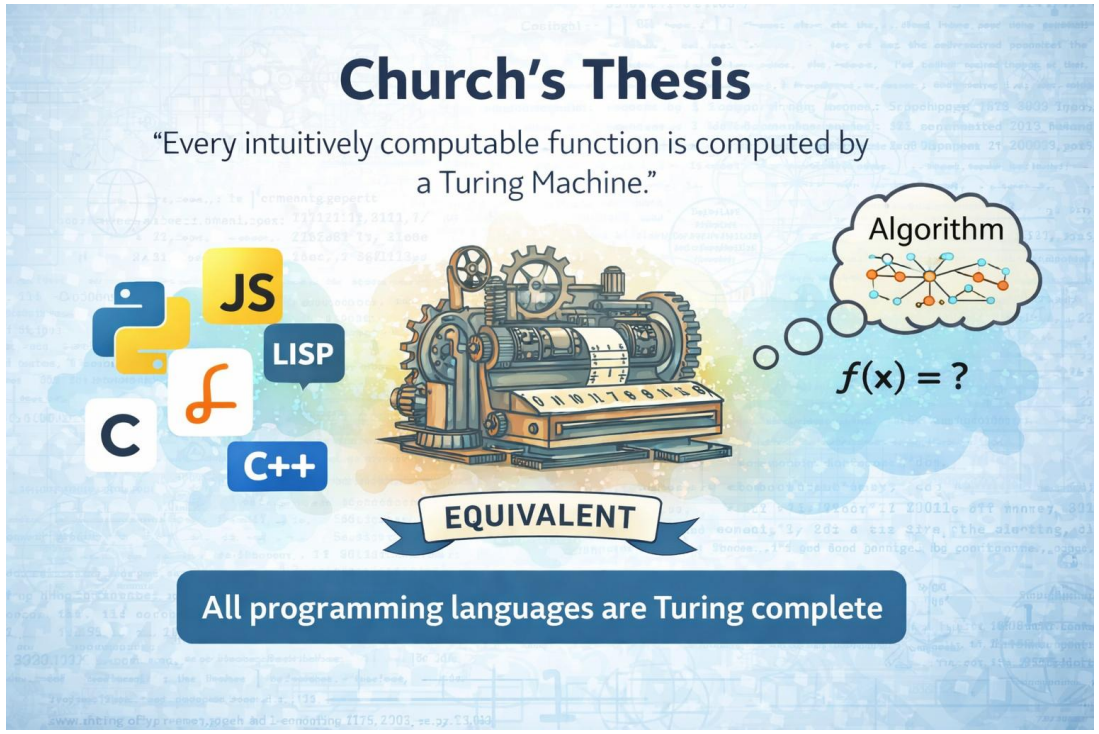
Kısıtlı Halting Problemi

- Şimdi varsayalım ki: “Bir program sabit fonksiyon hesaplıyor mu?” sorusu karar verilebilir olsun.
- Bu durumda: Verilen herhangi bir P için Q_p’yu oluştururuz. Q_p’nun sabit fonksiyon hesaplayıp hesaplamadığını test ederiz. Eğer sabitse $\Rightarrow$ P(P) sonlanır. Değilse $\Rightarrow$ P(P) sonlanmaz.
- Bu da **kısıtlı durma problemini (restricted halting problem)** çözmek anlamına gelir.
- Ancak durma probleminin karar verilemez olduğu bilinmektedir. Dolayısıyla başlangıç varsayımımız yanlıştır.

Hesaplanabilirlik

Church–Turing tezi

- Çeşitli programlama dillerinin (ve hesaplanabilirlik biçimselliklerinin) eşdeğerliğine ilişkin ispatlar gerçek teoremlerdir. İki dil L ve L′ verildiğinde, önce L içinde L′′nin yorumlayıcısı, sonra L′ içinde L'nin yorumlayıcısı yazılır. Böylece L ile L′′nin eşdeğer olduğu gösterilir. Bu tür ispatlar mevcut tüm diller için yapılmıştır; dolayısıyla fonksiyonel güç açısından eşdeğerlikleri kanıtlanmıştır.
- Bununla birlikte, bu durum kuramsal olarak, bir gün sezgisel olarak hesaplanabilir olup da mevcut hiçbir programlama diliyle ifade edilemeyen bir fonksiyon bulunabileceği ihtimalini tamamen dışlamaz. Ancak son 70 yılı aşkın süredir elde edilen tüm eşdeğerlik sonuçları, bunun mümkün olmadığına dair güçlü kanıt sunmaktadır.
- 1930'ların ortasında Alonzo Church bu imkânsızlığı ilke olarak ortaya koymuştur. Buna göre: **Sezgisel olarak hesaplanabilir her fonksiyon bir Turing Makinesi tarafından hesaplanabilir.**
- Eşdeğerlik sonuçlarının aksine bu ilke bir teorem değildir; çünkü "sezgisel hesaplanabilirlik" kavramı biçimsel olarak tanımlanabilir değildir. Bu nedenle daha çok felsefi bir ilke niteliğindedir. Bilgisayar bilimi topluluğu bu tezi güçlü biçimde benimser; hatta kuantum hesaplama gibi yeni paradigma önerilerinin bile onu geçersiz kılamayacağı varsayılır.



Hesaplanabilirlik

- Bilgisayar Bilimi'nin dayandığı hesaplama olgusu, algoritmaların hangi biçimsellikler içinde ifade edilebileceğini ve bunların sınırlarını inceleyen hesaplanabilirlik kuramına dayanır. Bu bölüm, söz konusu kuramın yalnızca en temel sonucunu sunmuştur. Bu sonuç son derece önemlidir ve her bilgisayar bilimcisinin bilmesi gereken bir gerçektir. Tanıtılan başlıca kavramlar şunlardır:
- **Karar verilemezlik (Undecidability):** Programların birçok önemli özelliği algoritmik (mekanik) bir yöntemle belirlenemez; durma problemi (halting problem) bunlardan biridir.
- **Hesaplanabilirlik (Computability):** Bir fonksiyon, onu hesaplayan bir program varsa hesaplanabilirdir. Durma probleminin karar verilemezliği, hesaplanamayan fonksiyonların da var olduğunu gösterir.
- **Kısmi tanımlılık (Partiality):** Bir programın ifade ettiği fonksiyonlar bazı girdiler için tanımsız olabilir; bu durum, programın o girdilerde sonlanmamasına karşılık gelir.
- **Turing-Tamlık (Turing Completeness):** Her genel amaçlı programlama dili, bir Turing Makinesi'nin hesaplayabildiği fonksiyonların aynısını hesaplar.